

一、 计算机系统

- 1、 冯诺依曼计算机的硬件系统由运算器、存储器、控制器、输入设备、输出设备五部分组成。
- 2、 存储容量 = 存储单元个数 (MAR 反应) * 存储字长 (MDR 反应)
- 3、 主频: 机器内部主时钟的频率
- 4、 时钟周期: CPU 中最小的时间单位
- 5、 指令周期包含若干机器周期, 机器周期包含若干时钟周期
- 6、 指令周期 = 取指令时间 + 执行指令时间
- 7、 指令系统: 计算机中所有机器指令的集合。

二、 运算器

- 1、 真值: 生活中的数据, 带有正负号
- 2、 机器数
 - a) 原码: 真值的正负号换成 0、1
 - b) 反码: 正数等于原码, 负数把数值位按位取反
 - c) 补码: 正数等于原码, 负数把数值位按位取反, 末位+1
 - d) 变形补码: 两位符号位
 - e) 补码中数值位相同, 数值位越大数就越大
 - f) 定点小数中 $[-1]_{\text{补}} = 1.0000000$, 定点整数中 $[-128]_{\text{补}} = 10000000$
 - g) 移码: 正负号换成 1、0, 数值位和补码一致
- 3、 奇偶校验码: 在原码前加一位校验位, 使总 1 数为奇数、偶数
- 4、 $[A+B]_{\text{补}} = [A]_{\text{补}} + [B]_{\text{补}}$, $[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}}$
- 5、 加减法溢出判别方法
 - a) 上溢: 大于最大表示数, 下溢: 小于最小表示数
 - b) 单符号位: 结果本应是正数, 算出来负数, 或相反

c) 双符号位：结果两符号位不同

d) 进位判别：数值位向符号位的进位、符号位向更高位的进位，如果不同时进行，则溢出。

6、逻辑移位：无符号数，直接移动，空位补 0

7、算数移位：符号位不变，正数都补 0，负数原码都补 0，反码都补 1，补码左 0 右 1

8、原码乘法：

原码乘法：已知 $x = -0.1110$, $y = -0.1101$, 求 $[x \cdot y]_{\text{原}}$

解：∵ $x = -0.1110$, ∴ $[x]_{\text{原}} = 1.1110$, $x^* = 0.1110$, $x_0 = 1$
∵ $y = -0.1101$, ∴ $[y]_{\text{原}} = 1.1101$, $y^* = 0.1101$, $y_0 = 1$

部分积	乘数
0.0000	1 1 0 1
+ 0.1110	
0.1110	
+ 0.0111	0 1 1 0
+ 0.0000	
0.0111	0
0.0011	1 0 1 1
+ 0.1110	
1.0001	1 0
0.1000	1 1 0 1
+ 0.1110	
1.0110	1 1 0
0.1011	0 1 1 0

即 $x^* \cdot y^* = 0.10110110$
符号位 $x_0 \oplus y_0 = 0$
∴ $[x \cdot y]_{\text{原}} = 0.10110110$

9、原码除法（恢复余数法）：用 $[x]_{\text{补}} + [-y]_{\text{补}}$ （减去除数），结果为正商 1，结果为负商 0，还要加回除数。然后左移。（不考!）

已知 $x = -0.1011$, $y = -0.1101$, 求 $[x/y]_{\text{原}}$

解：∵ $x = -0.1011$, ∴ $[x]_{\text{原}} = 1.1011$, $x^* = 0.1011$, $x_0 = 1$
∵ $y = -0.1101$, ∴ $[y]_{\text{原}} = 1.1101$, $y^* = 0.1101$, $y_0 = 1$

$[-y^*]_{\text{补}} = 1.0011$

被除数(余数)	商
0.1011	
+ 1.0011	0
1.1110	
+ 0.1101	
0.1011	0
1.0110	
+ 1.0011	
0.1001	0 1
1.0010	
+ 1.0011	0 1 1
0.0101	
0.1010	0 1 1 0
+ 1.0011	
1.1101	
+ 0.1101	
0.1010	0 1 1 0
1.0100	
+ 1.0011	
0.0111	0 1 1 0 1

∴ $[x/y]_{\text{原}} = 0.1101$

$x_0 \oplus y_0 = 1 \oplus 1 = 0$

∴ $[x/y]_{\text{原}} = 0.1101$

- 10、**原码除法（加减交替法）**：用 $[x]_{\text{补}} + [-y]_{\text{补}}$ （减去除数），结果为正商 1 左移减除数，结果为负商 0 左移加除数。

已知 $x = -0.1011$, $y = -0.1101$, 求 $[\frac{x}{y}]_{\text{原}}$

解： $\because x = -0.1011, \therefore [x]_{\text{原}} = 1.1011, x^* = 0.1011, x_0 = 1$
 $\because y = -0.1101, \therefore [y]_{\text{原}} = 1.1101, y^* = 0.1101, y_0 = 1$
 $[-y^*]_{\text{补}} = 1.0011$

被除数(余数)	商
0.1011	
+ 1.0011	
1.1110	0
1.1100	0
+ 0.1101	01
0.1001	01
+ 1.0011	011
0.0101	011
0.1010	
+ 1.0011	0110
1.1101	0110
1.1010	
+ 0.1101	01101
0.0111	

$\therefore [\frac{x^*}{y^*}]_{\text{原}} = 0.1101$

$x_0 \oplus y_0 = 1 \oplus 1 = 0$

$\therefore [\frac{x}{y}]_{\text{原}} = 0.1101$

- 11、**浮点数规格化**：原码的尾数最高位是 1，补码尾数最高位和符号位相反
- 12、尾数 $-\frac{1}{2}$ 的补码 1.1000 不是规格化数，-1 的补码 1.000 是规格化数



- 14、IEEE754 标准的阶码使用移码表示，即阶码=指数+127

- 15、浮点数加减法运算

- 对阶：先求阶差，小阶对大阶
- 尾数求和（差）
- 规格化：未溢出则一直左规直到规格化，溢出则右规一次
- 舍入：0 舍 1 入，舍掉的是 0 则需末尾+1
- 溢出判断：阶码溢出才是真的溢出

设 $x = 2^{-101} \times (-0.101000)$, $y = 2^{-100} \times (+0.111011)$
 并假设阶符取2位, 阶码的数值部分取3位
 数符取2位, 尾数的数值部分取6位, 求 $x-y$

解: $\therefore x = 2^{-101} \times (-0.101000)$, $y = 2^{-100} \times (+0.111011)$

$\therefore [x]_{补} = 11, 011; 11.011000$, $[y]_{补} = 11, 100; 00.111011$

① 对阶:

$$[\Delta_j]_{补} = [j_x]_{补} - [j_y]_{补} = 11, 011 + 00, 100 = 11, 111$$

即 $\Delta_j = -1$, 则 x 的尾数右移一位, 阶码 +1

$$[x]_{补} = 11, 100; 11.101100$$

② 求和:

$$\begin{aligned} [S_x]_{补}' - [S_y]_{补}' &= [S_x]_{补}' + [-S_y]_{补}' \\ &= 11.101100 + 11.000101 \\ &= 10.110001 \end{aligned}$$

$$\text{即 } [x-y]_{补} = 11, 100; 10.110001$$

③ 规格化:

两符号位不同, 需右规:

$$[x-y]_{补} = 11, 101; 11.011000$$

④ 舍入:

右规时末位丢1, 则有

$$\begin{array}{r} 11.011000 \\ + \quad \quad 1 \\ \hline 11.011001 \end{array}$$

$$\therefore [x-y]_{补} = 11, 101; 11.011001$$

⑤ 溢出判断

阶码未溢出, $\therefore$ 结果未溢出。

三、存储器

- 1、存储器设置层次结构是为了解决高速度、大容量、低价格三者的矛盾。
- 2、其中缓存-主存层次是为了解决 CPU 和主存速度不匹配的问题, 主存-辅存层次是为了解决存储系统的容量问题。
- 3、存储容量 = 存储单元个数 * 存储字长, 按字节编址、按字 (机器字 32 位/64 位) 编址、按双字编址, 按... 编址意思是存储字长是多少, 可以计算存储单元数。
- 4、静态随机存储器 (SRAM) 利用触发器存储信息, 存取快, 集成度低, 功耗大, 价格贵, 一般用于高速缓冲存储器。
- 5、动态随机存储器 (DRAM) 利用电容存储信息, 存取慢, 易于集成, 功耗低, 速度慢, 容量大, 一般用于大容量的主存储器, 需要刷新。
- 6、位扩展: 增加存储字长; 字扩展: 增加存储单元数量

7、 字位同时扩展

- 地址线：低位连芯片，高位做片选
- 数据线：相等直接连，不等按片接
- 读/写控制线：直连
- 片选线：重点

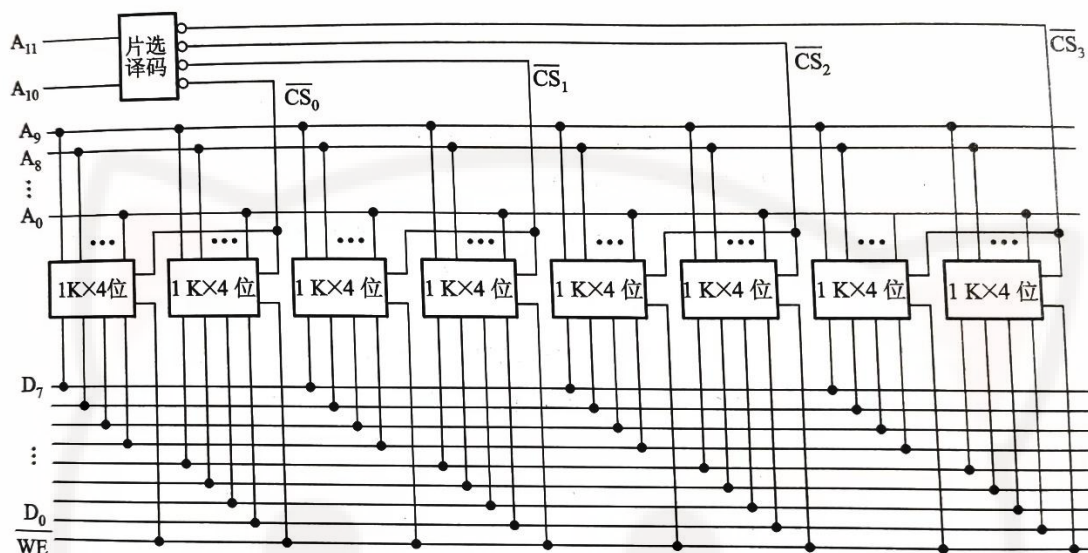


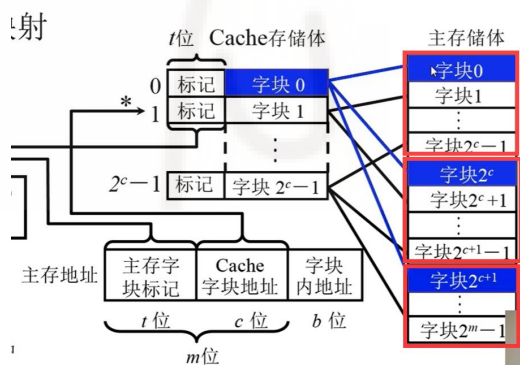
图 4.35 由 8 片 1 K×4 位的芯片组成 4 K×8 位的存储器

8、 给出地址范围 A~B，则存储容量=B-A+1

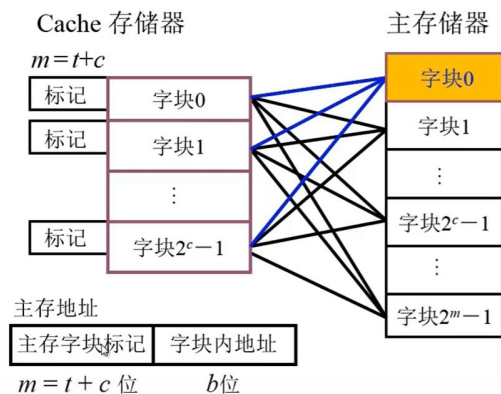
9、 Cache 写直达法：写操作时既写入 Cache 又写入主存

10、 Cache 写回法：写操作时只把数据写入 Cache，替换时才写回主存

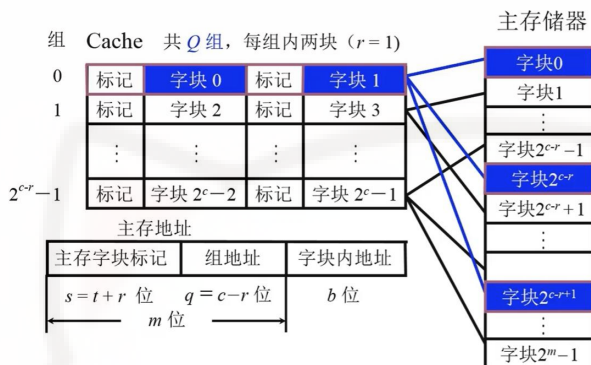
11、 直接相联（主存字块标记意思是区号，第几个红区）



12、 全相联（除字块内地址，剩下都存到标记）



13、组相联



14、产生全部指令的微操作命令序列的是 CU

15、多层次结构的计算机系统中硬件研究的主要对象为传统机器 M1 级和微程序机器 M0 级

16、简答 Cache 的工作原理。

Cache 是计算机系统的一种高速缓存存储器, 用于加快 CPU 访问主存储器的速度。它通过存储最近被访问的数据和指令, 以及它们相邻的数据和指令, 以提供更快的访问速度。当 CPU 需要访问数据或指令时, 它首先检查 Cache 中是否存在, 如果存在则直接从 Cache 中获取, 避免了访问主存储器的时间延迟。如果 Cache 中不存在所需的数据或指令, 那么 CPU 将从主存储器中获取并将其存储到 Cache 中, 以备将来的访问使用。这样, Cache 利用了局部性原理, 提高了计算机系统的整体性能。

四、 中央处理器 CPU

- 1、 控制单元外特性的输入：时钟、指令寄存器、标志、来自系统总线的控制信号
- 2、 输出：CPU 内的控制信号、送往系统总线的信号
- 3、 微指令的编码方式
 - a) 直接编码：每一位代表一个微命令
 - b) 字段直接编码：互斥的信号组成小组，用译码器译码。每个小组还要留一个状态，表示不发出命令
- 4、 CPU 功能：指令控制、操作控制、时间控制、数据加工
- 5、 从计算机的硬件组成来看

1)存储系统是怎样提高计算机运行速度的？

2)CPU 是怎样提高计算机运行速度的？

请分别举一个例子说明

答：1) 存储系统通过使用高速缓存存储器（Cache）来提高计算机运行速度。Cache 存储了最近被访问的数据和指令，以及它们相邻的数据和指令，使得 CPU 能够更快地获取所需的数据和指令。这样，CPU 在执行指令时不需要每次都从主存储器中读取数据，而是首先检查 Cache 中是否存在，从而减少了访问主存储器的时间延迟，提高了计算机的运行速度。

2) CPU 通过提高时钟频率来提高计算机运行速度。例如，当 CPU 的时钟频率增加时，它可以在同样的时间内执行更多的指令。这意味着 CPU 可以更快地完成任务，提高计算机的运行速度。举个例子，如果一个 CPU 的时钟频率从 2.4GHz 增加到 3.0GHz，它可以在同样的时间内执行更多的计算操作，从而加快计算机的运行速度。